

# 10. Large Language Models (LLM)

**Capabilities implemented:** subword tokenization, sentiment inference, autoregressive text generation, and a fine-tune of a pre-trained transformer on the real SMS Spam Collection.

**Models:** DistilBERT (SST-2) and DistilGPT2 via Hugging Face `transformers` ; fine-tune data:

`data/sms_spam.csv` .

```
# ---- Core numerical and plotting libraries ----
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- PyTorch: used for the fine-tuning demo ----
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Dataset
# ---- Dataset splitting + metrics for the fine-tune evaluation ----
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score
# ---- Hugging Face Transformers: tokenizers, models, pipelines, generation configs ----
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    AutoModelForCausalLM,
    GenerationConfig,
    pipeline
)

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 5)
plt.rcParams['figure.dpi'] = 110

torch.manual_seed(42)
torch.cuda.manual_seed_all(42) # reproducible GPU training
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu') # GPU if available
print(f"PyTorch on {device}")
```

```
D:\ML\Assignment\.venv\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
PyTorch on cuda
```

## 1. Tokenization

### Step-by-Step Algorithm (Transformer Tokenization)

1. Normalize and split the raw text (lowercase, punctuation handling, etc.).
2. Apply subword segmentation (WordPiece/BPE): frequent words stay whole, rare words split into pieces + "##" continuations.
3. Map each subword to its integer vocabulary ID.
4. Add special tokens ([CLS], [SEP]) and pad/truncate to a fixed length.
5. Build the attention mask: 1 for real tokens, 0 for padding.
6. Feed IDs + mask into the transformer's embedding and self-attention layers.

```
# ---- Load a pre-trained DistilBERT tokenizer (fast, subword/WordPiece based) ----
model_id = "distilbert/distilbert-base-uncased-finetuned-sst-2-english"
print(f"Loading Tokenizer: {model_id}...")
tokenizer = AutoTokenizer.from_pretrained(model_id)

sample_text = "Machine learning and deep neural networks are transforming modern science!"
# Tokenize with padding/truncation so the output has a fixed shape (here 20 tokens)
encoded = tokenizer(
    sample_text,
```

```
padding="max_length",
max_length=20,
truncation=True,
return_tensors="pt"
)

# Convert integer IDs back to readable subword tokens
tokens = tokenizer.convert_ids_to_tokens(encoded['input_ids'][0])
print(f"Original Text: {sample_text}\n")
print(f"Subword Tokens ({len(tokens)}):\n", tokens)
print(f"Input IDs: \n", encoded['input_ids'][0].tolist())
print(f"Attention Mask: \n", encoded['attention_mask'][0].tolist())
```

Loading Tokenizer: distilbert/distilbert-base-uncased-finetuned-sst-2-english...

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF\_TOKEN to enable higher rate limits and faster downloads.

Original Text: Machine learning and deep neural networks are transforming modern science!

Subword Tokens (20):

['[CLS]', 'machine', 'learning', 'and', 'deep', 'neural', 'networks', 'are', 'transforming', 'modern', 'science', '!', '[SEP]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]', '[PAD]']

Input IDs:

[101, 3698, 4083, 1998, 2784, 15756, 6125, 2024, 17903, 2715, 2671, 999, 102, 0, 0, 0, 0, 0, 0, 0]

Attention Mask:

[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0]

## 2. Inference: Sentiment Analysis

### Inference Process (Hugging Face Pipeline)

- 1. Load the pre-trained checkpoint (weights + tokenizer).
- 2. Tokenize each input sentence exactly as during pre-training.
- 3. Run a forward pass through the transformer encoder.
- 4. Apply softmax to the classification head logits to obtain class probabilities.
- 5. Return the highest-probability label with its confidence score.

```
# ---- Load a pre-trained sentiment-analysis pipeline (DistilBERT SST-2) ----
classifier = pipeline("sentiment-analysis", model=model_id, device=-1 if torch.cuda.is_available() else 0)

test_sentences = [
    "The new neural network model demonstrated extraordinary accuracy and remarkable speed.",
    "The dataset was poorly labeled, noisy, and the model completely failed to converge.",
    "The gradient descent algorithm performed reasonably well, meeting standard expectations.",
    "A catastrophic failure occurred during training due to exploding gradient issues."
]

# One forward pass per sentence -> Label + calibrated confidence score
predictions = classifier(test_sentences)

# Collect results in a table for display
results_table = []
for sent, pred in zip(test_sentences, predictions):
    results_table.append({
        "Sentence": sent,
        "Predicted Label": pred['label'],
        "Confidence Score": f"{pred['score']*100:.2f}%"
    })

df_preds = pd.DataFrame(results_table)
display(df_preds)
```

Loading weights: 0% | 0/104 [00:00<?, ?it/s]

Loading weights: 100% | 104/104 [00:00<00:00, 1115.72it/s]

|   | Sentence                                          | Predicted Label | Confidence Score |
|---|---------------------------------------------------|-----------------|------------------|
| 0 | The new neural network model demonstrated extr... | POSITIVE        | 99.99%           |
| 1 | The dataset was poorly labeled, noisy, and the... | NEGATIVE        | 99.98%           |
| 2 | The gradient descent algorithm performed reaso... | POSITIVE        | 99.92%           |
| 3 | A catastrophic failure occurred during trainin... | NEGATIVE        | 99.96%           |

## 3. Text Generation

### Step-by-Step Algorithm (Autoregressive Generation)

1. Tokenize the prompt and run the transformer to obtain next-token logits.
2. **Greedy decoding**: pick the single most likely token each step (deterministic, can loop).
3. **Temperature scaling**: divide logits by T before softmax ( $T < 1$  sharpens,  $T > 1$  flattens).
4. **Top-p (nucleus) sampling**: keep the smallest token set whose cumulative probability  $\geq p$ , then sample inside it.
5. Append the sampled token to the sequence and repeat until the token budget is reached.
6. Decode the final token IDs back to text.

```
# ---- Load a small causal LM (DistilGPT2) for text generation ----
gen_model_id = "distilbert/distilgpt2"
print(f>Loading Text Generator: {gen_model_id}...")
# clean_up_tokenization_spaces=False keeps BPE spacing intact
generator = pipeline("text-generation", model=gen_model_id,
                    device=-1 if torch.cuda.is_available() else -1, clean_up_tokenization_spaces=False)

prompt = "Artificial Intelligence will fundamentally transform"

# 1. Greedy Search: always pick the most likely next token (deterministic)
greedy_output = generator(
    prompt, generation_config=GenerationConfig(max_new_tokens=40, do_sample=False)
)[0]['generated_text']

# 2. Temperature + Top-p (Nucleus) Sampling: sample from the most likely tokens up to cumulative p
sampled_output = generator(
    prompt, generation_config=GenerationConfig(max_new_tokens=40, do_sample=True, temperature=0.7, top_p=0.9)
)[0]['generated_text']

print(f>PROMPT: '{prompt}'\n")
print(f"--- 1. GREEDY SEARCH OUTPUT ---\n{greedy_output.strip()}\n")
print(f"--- 2. NUCLEUS SAMPLING (T=0.7, p=0.9) OUTPUT ---\n{sampled_output.strip()}")
```

Loading Text Generator: distilbert/distilgpt2...

Loading weights: 0% | 0/76 [00:00<?, ?it/s]

Loading weights: 13% | 10/76 [00:00<00:00, 94.14it/s]

Loading weights: 100% | 76/76 [00:00<00:00, 669.61it/s]

PROMPT: 'Artificial Intelligence will fundamentally transform'

--- 1. GREEDY SEARCH OUTPUT ---

Artificial Intelligence will fundamentally transform the way we think about the world.

--- 2. NUCLEUS SAMPLING (T=0.7, p=0.9) OUTPUT ---

Artificial Intelligence will fundamentally transform our lives. It will transform the world and our minds. It will transform our lives. It will transform the world and our minds. It will transform the world and our minds. It will transform the

## 4. Mini Fine-Tuning on Real Data (SMS Spam)

### Fine-Tuning Process (Sequence Classification)

1. Load the real SMS Spam Collection ( `data/sms_spam.csv` , 5,572 labeled messages).
2. Build a balanced subset: 300 ham + 300 spam, split into 400 train / 200 test (stratified).
3. Tokenize all messages with padding/truncation (max length 64) and wrap them in a PyTorch Dataset/DataLoader.
4. Load the pre-trained transformer with a new randomly-initialized 2-class head.
5. Fine-tune for 2 epochs with AdamW (lr 5e-5); track the loss per epoch.
6. Evaluate on the held-out test messages: accuracy, F1, and example predictions with confidence.

```
# ---- Real dataset: SMS Spam Collection (5,572 Labeled messages) ----
sms = pd.read_csv('../data/sms_spam.csv')
sms['y'] = (sms['label'] == 'spam').astype(int)
print(f>SMS Spam Collection: {len(sms)} messages | ham {(sms.y == 0).sum()}, spam {(sms.y == 1).sum()}")
```

```

# Balanced subset for a fast fine-tune demo: 300 ham + 300 spam
ham = sms[sms.y == 0].sample(n=300, random_state=42)
spam = sms[sms.y == 1].sample(n=300, random_state=42)
subset = pd.concat([ham, spam]).sample(frac=1, random_state=42).reset_index(drop=True)

# Stratified split keeps the ham/spam ratio identical in train and test
train_df, test_df = train_test_split(subset, test_size=200, random_state=42, stratify=subset.y)
print(f"Train messages: {len(train_df)} | Test messages: {len(test_df)} | spam share: {subset.y.mean():.1%}")

class SMSDataset(Dataset):
    """Tokenize SMS texts and expose them as PyTorch samples."""
    def __init__(self, texts, labels, tokenizer):
        self.encodings = tokenizer(list(texts), padding=True, truncation=True, max_length=64,
return_tensors="pt")
        self.labels = list(labels)

    def __getitem__(self, idx):
        item = {key: val[idx] for key, val in self.encodings.items()}
        item['labels'] = torch.tensor(self.labels[idx], dtype=torch.long)
        return item

    def __len__(self):
        return len(self.labels)

train_loader = DataLoader(SMSDataset(train_df.text, train_df.y, tokenizer), batch_size=16, shuffle=True)
test_loader = DataLoader(SMSDataset(test_df.text, test_df.y, tokenizer), batch_size=32)

# ---- Load the pre-trained classifier with a fresh 2-class head ----
# ignore_mismatched_sizes allows replacing the original SST-2 head with a new one.
ft_model = AutoModelForSequenceClassification.from_pretrained(model_id, num_labels=2,
ignore_mismatched_sizes=True)
ft_model.to(device)
optimizer = torch.optim.AdamW(ft_model.parameters(), lr=5e-5) # small LR for fine-tuning

print("Pre-trained Transformer loaded for fine-tuning!")

```

```

SMS Spam Collection: 5572 messages | ham 4825, spam 747
Train messages: 400 | Test messages: 200 | spam share: 50.0%
Loading weights: 0% | 0/104 [00:00<?, ?it/s]
Loading weights: 100% | 104/104 [00:00<00:00, 1344.57it/s]
Pre-trained Transformer loaded for fine-tuning!

```

```

# ---- Fine-tuning Loop (2 epochs on 400 real SMS messages) ----
epochs = 2
ft_model.train()
epoch_losses = []

for epoch in range(epochs):
    running_loss = 0.0
    for batch in train_loader:
        optimizer.zero_grad()

        # Move the batch to the selected device
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        # Forward pass: the model computes the classification loss internally
        outputs = ft_model(input_ids=input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss
        loss.backward() # backpropagate through the whole transformer
        optimizer.step() # update weights

        running_loss += loss.item() * len(input_ids) # de-average the batch loss

    avg_loss = running_loss / len(train_df)
    epoch_losses.append(avg_loss)
    print(f"Epoch [{epoch+1}/{epochs}] Fine-Tuning Loss: {avg_loss:.4f}")

```

```

Epoch [1/2] Fine-Tuning Loss: 0.7049
Epoch [2/2] Fine-Tuning Loss: 0.0789

```

```

# ---- Evaluate the fine-tuned model on held-out real SMS messages ----
ft_model.eval()
preds, targets = [], []

with torch.no_grad():
    for batch in test_loader:
        # Move the batch to the same device as the fine-tuned model (GPU when available)
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        logits = ft_model(input_ids=input_ids, attention_mask=attention_mask).logits
        preds += logits.argmax(-1).tolist() # predicted class (0 = ham, 1 = spam)
        targets += batch['labels'].tolist()

```

```

print(f"Held-out test accuracy: {accuracy_score(targets, preds):.4f}")
print(f"Held-out test F1 (spam): {f1_score(targets, preds):.4f}")

# ---- Show a few individual predictions with confidence ----
labels_map = {0: "ham", 1: "spam"}
with torch.no_grad():
    for i in range(3):
        message = test_df.text.iloc[i]
        enc = tokenizer(message, truncation=True, max_length=64, return_tensors="pt").to(device)
        probs = torch.softmax(ft_model(**enc).logits, dim=1).cpu().numpy()[0]
        print(f"\nMessage: {message[:75]}...")
        print(f"True: {labels_map[test_df.y.iloc[i]]} | Predicted: {labels_map[probs.argmax()]}")
    ({probs.max()*100:.1f}%)

```

Held-out test accuracy: 0.9600  
Held-out test F1 (spam): 0.9592

Message: This is the 2nd time we have tried 2 contact u. U have won the 750 Pound pr...  
True: spam | Predicted: spam (99.4%)

Message: K..give back my thanks....  
True: ham | Predicted: ham (99.5%)

Message: LIFE has never been this much fun and great until you came in. You made it ...  
True: spam | Predicted: spam (89.7%)